

CONFIGURAÇÃO DE AMBIENTE

Lista de exercícios — Variáveis de Ambiente com dotenv

Carregamento, separação de ambientes, segurança no Git e validação

Atividade prática

Esta lista contém cinco exercícios. Os códigos possuem falhas intencionais e os trechos de apoio não entregam a resolução completa. O tema comum é a configuração que parece certa porque funciona na máquina de quem escreveu: o valor existe no arquivo, mas chega vazio, chega como texto, chega tarde demais ou não chega ao servidor.

Exercícios

- 1 - Criando e usando o arquivo .env.
- 2 - Separando ambientes de desenvolvimento e produção.
- 3 - Segurança no Git e arquivo de exemplo.
- 4 - Validação de variáveis obrigatórias.
- 5 - Porta com valor padrão e teste em duas instâncias.

Em cada exercício, registre o sintoma, sua hipótese, a evidência encontrada e a correção aplicada. Aqui a evidência costuma ser o valor real de uma variável no momento exato em que ela é lida: imprima ou inspecione esse valor em vez de supor o que está nele.

Pré-requisitos

```
npm init -y
npm install express dotenv
npm install -D typescript ts-node @types/node @types/express
npx tsc --init

# usados a partir do exercício 2
npm install -D cross-env
npm install env-schema          # ou: npm install joi

git init
```

O projeto pode ser novo ou o mesmo das listas anteriores. Os exercícios são encadeados: o arquivo criado no exercício 1 é reaproveitado até o fim.

Orientações de investigação

Quase todo defeito de configuração se resolve olhando três coisas: qual arquivo foi lido, em que momento ele foi lido e qual valor de fato chegou em `process.env`.

Etapa	O que fazer
1. Reproduzir	Rodar o comando exato que falha, sem alterar o ambiente antes.
2. Observar	Imprimir o valor lido e o seu tipo, não apenas confirmar que a chave existe.
3. Formular hipótese	Apontar se o problema é o arquivo, a ordem de carregamento ou o tipo.
4. Investigar	Conferir caminho do arquivo, ordem dos imports e precedência do ambiente.
5. Corrigir	Aplicar a menor mudança que resolva a causa.
6. Verificar	Repetir em pelo menos dois ambientes e com a variável ausente.

Registro obrigatório

Sintoma:
Comando executado:
Resultado esperado:
Resultado obtido (valor e tipo da variável lida):
Hipótese:
Como foi investigado:
Causa encontrada:
Como a correção foi validada:

Regras

- Não versionar o arquivo `.env` nem qualquer segredo real.
- Não imprimir `process.env` inteiro nem o valor de chaves e senhas.
- Não usar valor padrão para segredo: ausência de `API_KEY` é erro, não é 'vazio'.
- Não ler `process.env` espalhado pelo código, apenas no módulo de configuração.
- Não confiar em variável de ambiente sem validar tipo e presença.
- Não deixar a aplicação subir com configuração inválida.

Avaliação: o raciocínio usado para encontrar a falha vale tanto quanto o resultado final.

Exercício 1 - Criando e Usando .env

O arquivo .env foi criado com PORT e DB_HOST, mas a aplicação imprime undefined para as duas variáveis. Depois de mexer na ordem dos imports, DB_HOST passa a aparecer, só que com aspas no meio do valor, e a porta continua se comportando como texto.

```
# .env
PORT = 3000
DB_HOST = "localhost"
DB_PASSWORD=senha123
```

```
// src/database.ts
export const host = process.env.DB_HOST;
console.log('host do banco:', host);

// src/app.ts
import { host } from './database';
import dotenv from 'dotenv';

dotenv.config();

console.log('variaveis carregadas:', process.env);
console.log('porta + 1:', process.env.PORT + 1);
```

Tarefas

- Executar como está e copiar exatamente o que foi impresso.
- Explicar por que database.ts lê a variável antes de ela existir.
- Corrigir a ordem de carregamento para que dotenv rode antes de qualquer leitura.
- Explicar o que os espaços e as aspas no arquivo produzem no valor final.
- Imprimir o tipo de process.env.PORT e explicar o resultado de 'porta + 1'.
- Converter a porta para número em um único lugar do projeto.
- Substituir a impressão de process.env por uma impressão segura e seletiva.
- Centralizar a leitura das variáveis em um módulo de configuração exportado.

Pistas graduais

- Lembre que os imports de um módulo são executados antes do corpo do arquivo que importa.
- Compare o valor de DB_HOST com e sem espaços ao redor do sinal de igual.
- Verifique o que typeof devolve para qualquer entrada de process.env.
- Observe o que a soma faz quando um dos operandos é texto.
- Procure o que dotenv faz quando a chave já existe no ambiente do sistema.

Exercício 1 - Apoio e verificação

Trecho de apoio

```
// src/config.ts - único lugar do projeto que lê process.env
import 'dotenv/config'; // ou dotenv.config(), antes de qualquer outro import

function required(name: string): string {
  const value = process.env[name];
  /* o que fazer quando o valor está ausente ou vazio? */
}

export const config = {
  port: /* converta para número e trate entrada inválida */,
  dbHost: required('DB_HOST')
};

console.log('config carregada', { port: config.port, dbHost: config.dbHost });
```

Cenários mínimos

Cenário	O que verificar
Arquivo .env completo	Os dois valores aparecem limpos, sem aspas nem espaços.
Variável PORT ausente	Comportamento definido e previsível, não undefined silencioso.
PORT definida no terminal	Qual valor vence: o do sistema ou o do arquivo.
Valor com espaço ao redor	O valor final chega sem os espaços.
Impressão da configuração	Nenhuma senha ou chave aparece no console.

Perguntas orientadoras

- Por que toda variável de ambiente chega como texto?
- O que acontece se dotenv for carregado depois de um módulo que já leu a variável?
- Por que imprimir process.env inteiro é um risco, mesmo em desenvolvimento?
- Qual a vantagem de um módulo único de configuração sobre process.env espalhado?
- Por que o arquivo .env não deve conter comentários com valores reais antigos?

Critério: a aplicação precisa ler cada variável uma única vez, com tipo correto e sem expor segredo no console.

Exercício 2 - Separando Ambientes

Os dois scripts foram criados, mas ambos sobem com a mesma configuração. Em produção a aplicação conecta no banco de desenvolvimento. Em uma máquina Windows, nenhum dos dois scripts chega a rodar.

```
// package.json
"scripts": {
  "start:dev": "NODE_ENV=development ts-node src/app.ts",
  "start:prod": "NODE_ENV=production ts-node src/app.ts"
}
```

```
// src/app.ts
import dotenv from 'dotenv';

dotenv.config();

const env = process.env.NODE_ENV;

if (env = 'production') {
  dotenv.config({ path: '.env.prod' });
} else {
  dotenv.config({ path: '../.env.dev' });
}

console.log('ambiente:', env, 'banco:', process.env.DB_HOST);
```

Tarefas

- Criar .env.dev e .env.prod com valores distintos de DB_HOST.
- Rodar os dois scripts e mostrar que o banco impresso é o mesmo.
- Encontrar e explicar o erro na condição que escolhe o ambiente.
- Explicar por que o primeiro dotenv.config() atrapalha os seguintes.
- Corrigir o caminho do arquivo para não depender do diretório de execução.
- Fazer os scripts funcionarem também no Windows.
- Definir o comportamento quando NODE_ENV não está definida.
- Mostrar, com evidência, que cada script carrega o arquivo correspondente.

Pistas graduais

- Releia a condição do if com atenção: há uma atribuição onde deveria haver comparação.
- Descubra o que dotenv faz por padrão com uma chave que já está definida.
- Compare o caminho relativo usado com o diretório em que o comando foi disparado.
- Investigue por que a sintaxe VAR=valor antes do comando não funciona no Windows.
- Verifique se NODE_ENV já está definida antes de o arquivo ser lido.

Exercício 2 - Apoio e verificação

Trecho de apoio

```
import path from 'node:path';
import dotenv from 'dotenv';

const env = process.env.NODE_ENV ?? /* qual padrão é seguro aqui? */;

dotenv.config({
  path: path.resolve(/* ancore em um ponto fixo do projeto */, `.${env}`),
  override: /* o arquivo deve vencer o ambiente do sistema? justifique */
});
```

```
// package.json - forma portátil
"scripts": {
  "start:dev": "cross-env NODE_ENV=development ts-node src/app.ts",
  "start:prod": "cross-env NODE_ENV=production ts-node src/app.ts"
}
```

Cenários mínimos

Execução	O que verificar
npm run start:dev	DB_HOST é o valor de .env.dev.
npm run start:prod	DB_HOST é o valor de .env.prod.
Sem NODE_ENV definida	Comportamento padrão definido e documentado.
NODE_ENV inválida	Falha clara na subida, não um arquivo inexistente ignorado.
Comando rodado de outra pasta	O arquivo correto continua sendo encontrado.

Perguntas orientadoras

- Qual a diferença entre um valor vindo do arquivo e um vindo do ambiente do sistema?
- Por que em produção normalmente não existe arquivo .env?
- O que o operador de atribuição dentro de um if devolve como resultado?
- Por que ancorar o caminho do arquivo é melhor que usar um caminho relativo?
- Quando faz sentido que o arquivo sobrescreva uma variável já definida?

Critério: cada script precisa provar, pela saída, qual arquivo foi carregado.

Exercício 3 - Segurança no Git

O .gitignore foi configurado, mas o git continua mostrando o .env como modificado. Ao mesmo tempo, o arquivo de exemplo desapareceu do repositório e um colega abriu um chamado dizendo que não consegue descobrir quais variáveis precisa definir.

```
# .gitignore
.env*
node_modules
```

```
# .env.example (versionado)
PORT=3000
DB_HOST=db.producao.interno
DB_PASSWORD=senha123
API_KEY=sk_live_9f2a8c1d4b7e
```

```
$ git status
modified:   .env

$ git check-ignore -v .env.example
.gitignore:1:.env*      .env.example
```

Tarefas

- Explicar por que o .env continua aparecendo no git status mesmo estando no .gitignore.
- Remover o arquivo do rastreamento sem apagá-lo do disco.
- Ajustar o padrão para ignorar .env e .env.dev, mas manter .env.example versionado.
- Comprovar o resultado com git check-ignore nos três arquivos.
- Reescrever o .env.example contendo apenas nomes e valores de exemplo.
- Explicar o que fazer com a chave que já foi enviada ao repositório.
- Descrever como um colega deve criar o próprio .env a partir do exemplo.
- Verificar se algum segredo aparece no histórico com git log de busca.

Pistas graduais

- Descubra por que o .gitignore não afeta arquivos que o git já rastreia.
- Procure o comando que remove do índice mantendo o arquivo local.
- Estude o caractere de negação em padrões do .gitignore.
- Pense no que continua acessível no histórico mesmo após apagar o arquivo.
- Considere que um segredo exposto precisa ser rotacionado, não só removido.

Exercício 3 - Apoio e verificação

Trecho de apoio

```
# .gitignore
.env
.env.*
/* como abrir exceção para o arquivo de exemplo? */

# parar de rastrear sem apagar do disco
git rm /* qual opção mantém o arquivo local? */ .env
git commit -m "remove .env do versionamento"

# conferir cada caso
git check-ignore -v .env .env.dev .env.example
```

```
# .env.example - apenas nomes e valores ilustrativos
PORT=3000
DB_HOST=localhost
DB_PASSWORD=
API_KEY=
```

Cenários mínimos

Verificação	Resultado esperado
.env	Ignorado e fora do índice do git.
.env.dev e .env.prod	Ignorados.
.env.example	Versionado e sem nenhum valor real.
git status após a correção	Nenhum arquivo de ambiente listado.
Clone limpo do repositório	O projeto explica quais variáveis criar.

Perguntas orientadoras

- Por que adicionar ao .gitignore não basta para um arquivo já commitado?
- O que permanece no repositório mesmo depois de o arquivo ser apagado?
- Por que rotacionar a chave exposta é obrigatório, e não opcional?
- Qual o papel do arquivo de exemplo na entrada de um novo integrante?
- Como um segredo deve chegar ao servidor se o arquivo não é versionado?

Critério: nenhum valor real pode permanecer versionado, e o repositório precisa continuar utilizável por quem clona do zero.

Exercício 4 - Variáveis Obrigatórias

A validação foi adicionada, mas a aplicação sobe normalmente sem API_KEY e só quebra quando o primeiro cliente chama a rota de integração, meia hora depois do deploy. A porta validada também volta a ser texto em outro ponto do código.

```
import Joi from 'joi';

const schema = Joi.object({
  DB_HOST: Joi.string(),
  API_KEY: Joi.string(),
  PORT: Joi.number()
});

export function validate() {
  const result = schema.validate(process.env);

  if (result.error) {
    console.warn('configuração incompleta');
  }

  return process.env;
}

// src/integration.ts
export const apiKey = process.env.API_KEY;
```

Tarefas

- Rodar sem API_KEY e mostrar que a aplicação sobe assim mesmo.
- Explicar por que o schema atual aceita a ausência das variáveis.
- Tornar DB_HOST e API_KEY obrigatórias e falhar na subida quando faltarem.
- Exibir na mensagem de erro quais variáveis faltaram, sem mostrar valores.
- Usar o valor validado e convertido, em vez de voltar a ler process.env.
- Garantir que a porta chegue ao restante do código já como número.
- Permitir variáveis extras do sistema sem que a validação as rejeite.
- Refazer o exercício com env-schema e comparar as duas abordagens.

Trecho de apoio

```
const schema = Joi.object({
  DB_HOST: Joi.string()/* como exigir presença? */,
  API_KEY: Joi.string()/* idem, e como impedir string vazia? */,
  PORT: Joi.number()/* faixa válida e valor padrão */
}).unknown(/* o que fazer com as demais variáveis do sistema? */);

const { value, error } = schema.validate(process.env, { abortEarly: false });

if (error) {
  /* interrompa a subida deixando claro o que faltou */
}

export const config = value; // use este objeto, não process.env
```

Exercício 4 - Investigação e verificação

Pistas graduais

- Descubra o que um schema de string aceita quando a chave simplesmente não existe.
- Compare `console.warn` com uma interrupção real da inicialização.
- Verifique o que `abortEarly` muda na lista de erros devolvida.
- Observe que o objeto validado devolve os valores já convertidos.
- Pense no que acontece quando a validação roda, mas ninguém usa o resultado.

Cenários mínimos

Cenário	O que verificar
Todas as variáveis presentes	A aplicação sobe e a porta é número.
API_KEY ausente	A subida falha imediatamente, com mensagem clara.
API_KEY vazia	Tratada como ausente, não como valor válido.
DB_HOST e API_KEY ausentes	As duas faltas aparecem no mesmo erro.
PORT com texto inválido	Erro de validação, não porta silenciosamente errada.

Perguntas orientadoras

- Por que falhar na subida é melhor que falhar na primeira requisição?
- Qual a diferença entre variável ausente e variável definida como texto vazio?
- Por que a mensagem de erro não deve conter o valor da variável?
- Que risco existe em rejeitar variáveis desconhecidas do sistema?
- Que vantagem o objeto validado traz sobre continuar lendo `process.env`?

Critério: configuração incompleta precisa impedir a aplicação de subir, e o erro deve listar todas as faltas de uma vez.

Exercício 5 - Porta com Fallback

O servidor usa a porta do ambiente com valor padrão, mas ao definir PORT como texto vazio ele sobe em 3000 sem avisar. Ao tentar rodar duas instâncias para comparar, a segunda morre com um erro que ninguém lê porque não há tratamento.

```
import express from 'express';

const app = express();
const port = process.env.PORT || 3000;

app.get('/health', (_req, res) => {
  res.json({ ok: true, port });
});

app.listen(port, () => {
  console.log('servidor no ar');
});
```

Tarefas

- Subir sem definir PORT e registrar em qual porta o servidor atendeu.
- Subir com PORT=4000 e confirmar a mudança pela rota /health.
- Subir com PORT vazia e explicar por que o padrão foi aplicado.
- Comparar o comportamento do operador || com o operador ?? nesse caso.
- Converter a porta para número e recusar valor fora da faixa válida.
- Imprimir a porta real no callback do listen, em vez de uma mensagem fixa.
- Subir duas instâncias ao mesmo tempo e capturar o erro da segunda.
- Tratar o erro de porta ocupada com mensagem legível e saída controlada.

Trecho de apoio

```
const raw = process.env.PORT;
const port = /* converta, valide a faixa e decida o padrão */;

const server = app.listen(port, () => {
  const address = server.address();
  console.log('servidor no ar', /* mostre a porta realmente usada */);
});

server.on('error', (err: NodeJS.ErrnoException) => {
  if (err.code === /* qual código indica porta ocupada? */) {
    /* mensagem legível e encerramento controlado */
  }
});
```

Exercício 5 - Investigação e entrega

Pistas graduais

- Liste quais valores o operador `||` considera falsos, e veja se o texto vazio está entre eles.
- Verifique o que o Express faz ao receber a porta como texto em vez de número.
- Descubra o que `server.address()` devolve quando a porta é zero.
- Procure o código de erro emitido quando o endereço já está em uso.
- Observe que um erro de listen não é lançado, é emitido como evento.

Cenários mínimos

Execução	O que verificar
Sem PORT	Servidor sobe em 3000 e informa a porta real.
PORT=4000	Servidor sobe em 4000, confirmado pela rota <code>/health</code> .
PORT vazia	Comportamento definido e explicado, não acidental.
PORT=abc	Falha clara na subida, com mensagem sobre a porta.
Duas instâncias na mesma porta	A segunda falha com mensagem legível e código de saída.

Perguntas orientadoras

- Qual a diferença prática entre `||` e `??` para valores de configuração?
- Por que imprimir a porta obtida do servidor é melhor que imprimir a variável?
- Em que situação a porta deve vir do ambiente e não ter valor padrão?
- Por que um erro de inicialização deve encerrar o processo com código diferente de zero?

Checklist final

Exercício	Entrega mínima
1	<code>dotenv</code> carregado antes de qualquer leitura, módulo único de configuração, porta como número e nenhum segredo impresso.
2	Dois arquivos de ambiente, scripts portáteis, arquivo correto comprovado pela saída e padrão definido sem <code>NODE_ENV</code> .
3	<code>.env</code> fora do índice, <code>.env.example</code> versionado sem valor real, <code>git check-ignore</code> nos três arquivos e plano para a chave exposta.
4	Variáveis obrigatórias validadas na subida, todas as faltas em um erro e uso do objeto validado.
5	Porta convertida e validada, porta real impressa, cinco cenários testados e erro de porta ocupada tratado.

Entrega final: envie o projeto com os arquivos de ambiente de exemplo, o módulo de configuração e um pequeno registro explicando, por exercício, qual era a causa da falha e como ela foi comprovada.